

Lors de ce TP, sur 2 séances, nous allons développer une application web, coté client, qui utilise un service web

TP n°3 et 4

AJAX

Alexandre GUIDET

Compétence 1	Compétence 2	Compétence 4	Compétence 5
☒ Élaborer et implémenter les spécifications ☐ Appliquer des principes d'accessibilité	☐ Évaluer l'impact des solutions	☐ Assurer la confidentialité des données ☒ Organiser la restitution des données	☐ Formaliser les besoins
<i>En respectant les besoins décrits par le client</i> <i>En appliquant les principes algorithmiques</i> <i>En veillant à la qualité du code et à sa documentation</i> <i>En choisissant les ressources techniques appropriées</i>	<i>En formalisant et modélisant des situations complexes</i> <i>En recensant les algorithmes et les structures de données usuels</i> <i>En s'appuyant sur des schémas de raisonnement</i> <i>En justifiant les choix et validant les résultats</i>	<i>En respectant les réglementations sur le respect de la vie privée et la protection des données personnelles</i> <i>En respectant les enjeux économiques, sociétaux et écologiques de l'utilisation du stockage de données</i> <i>En s'appuyant sur des bases mathématiques</i> <i>En assurant la cohérence et la qualité</i>	<i>En communiquant efficacement avec les différents acteurs du projet</i> <i>En respectant les règles juridiques et les normes en vigueur</i> <i>En sensibilisant à une gestion éthique, responsable, durable et interculturelle</i> <i>En adoptant une démarche proactive, créative et critique</i>

1 INTRODUCTION

L'application web que nous souhaitons développer est un **client** pour un service web (très simple) fourni par l'IUT. Vous trouverez la **documentation** de ce service web dans le dossier TP3-base du commun.

Le client à développer sera un client riche, exécutable dans un navigateur. Il est susceptible d'être hébergé sur un autre serveur web que le webservice fourni mais ce n'est pas obligatoire.

Le client doit être développé en *TypeScript*. Une dépendance tierce doit être autorisée par l'enseignant.

Voici une maquette fonctionnelle du client demandé :



- Au départ, la partie « ajouter une personne » n'est pas visible. La liste des personnes est initialisée dès le lancement de l'application.
- Le clic sur « ajouter » rend la partie « ajouter une personne » visible.
- Le clic sur « valider » utilise les champs de saisie pour créer une personne. L'annuaire partagé est mis à jour, la liste également, et la partie « ajouter une personne » est cachée.
- Un clic sur une personne doit la sélectionner, la mettre en évidence.
- Le clic sur « téléphone » donne le numéro de téléphone de la personne sélectionnée dans une boîte de dialogue « *popup* » (fonction alert).
- Le clic sur « supprimer » supprime la personne sélectionnée de la liste et de l'annuaire partagé.

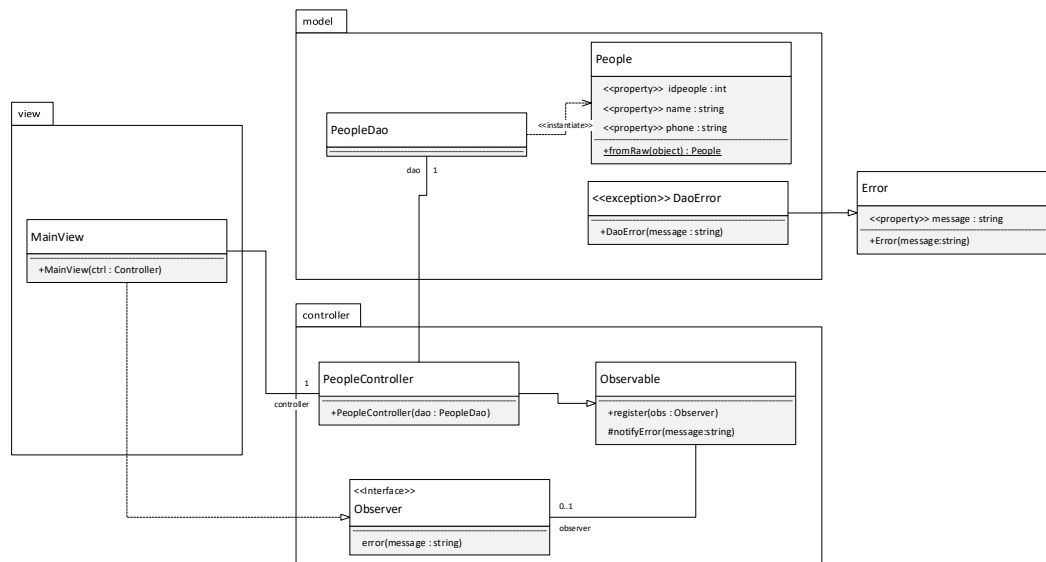
Créez un dépôt **privé** dans dept-info-iut-dijon appelé R410-TP3-<votre nom> et ajoutez votre enseignant à ce dépôt.

Récupérez le projet correspondant à la maquette depuis TP03-base (vous devez avoir, outre index.html et styles.css, un .gitignore, le tsconfig.json et un favicon). Dans la feuille de styles, vous avez quelques classes CSS qui peuvent vous servir

(hidden et selected par ex). Ajoutez ces fichiers à votre dépôt (dans la branche principale). Validez.

- ❶ *N'avancez pas dans le TP si vous n'avez pas une fonctionnalité terminée, il ne sert à rien de démarrer toutes les fonctionnalités si aucune ne fonctionne. Ne passez à la suite que si la fonctionnalité est ok, demandez à votre enseignant au besoin. Gérez bien vos branches : seule la branche principale sera corrigée, et seules ce qui fonctionne rapporte des points. Attention à la qualité du code : des malus seront affectés sur divers points : respect du CDC, qualité du code, documentation, etc.*
- ❶ *A chaque fonctionnalité, vous aurez un diagramme de classes et de séquence pour la décrire, suivis d'un texte qui détaille le travail à faire. Suivez bien le texte, ne partez pas sur le code sans avoir lu la question le détaillant !*

2 STRUCTURE DE BASE DU PROJET



Créez les dossiers, dans le dossier `ts`, correspondant aux *packages*. La classe `Error` est une classe standard et ne doit pas être codée.

Créez dans `model` la classe `People`, l'exception `DaoError` et la classe `PeopleDao` (elle sera vite pour l'instant).

Note : pour `People.fromRaw` utilisez le code suivant :

```

public static fromRaw(obj : any):People
{
    let people = new People();
    if(obj.idpeople) people._idpeople = obj.idpeople;
    if(obj.name) people._name = obj.name;
    if(obj.phone) people._phone = obj.phone;
    return people;
}

```

Créez dans `controller` les classes indiquées.

Créez dans view la classe `MainView`. Dans l'implémentation de `Observer.error`, faites simplement afficher le message par la fonction `alert`. N'oubliez pas que la vue (`MainView`) DOIT s'abonner au contrôleur pour en recevoir les notifications !

Créez enfin un fichier `main.ts`, dans le dossier `ts`, qui va simplement créer la vue et injecter les dépendances :

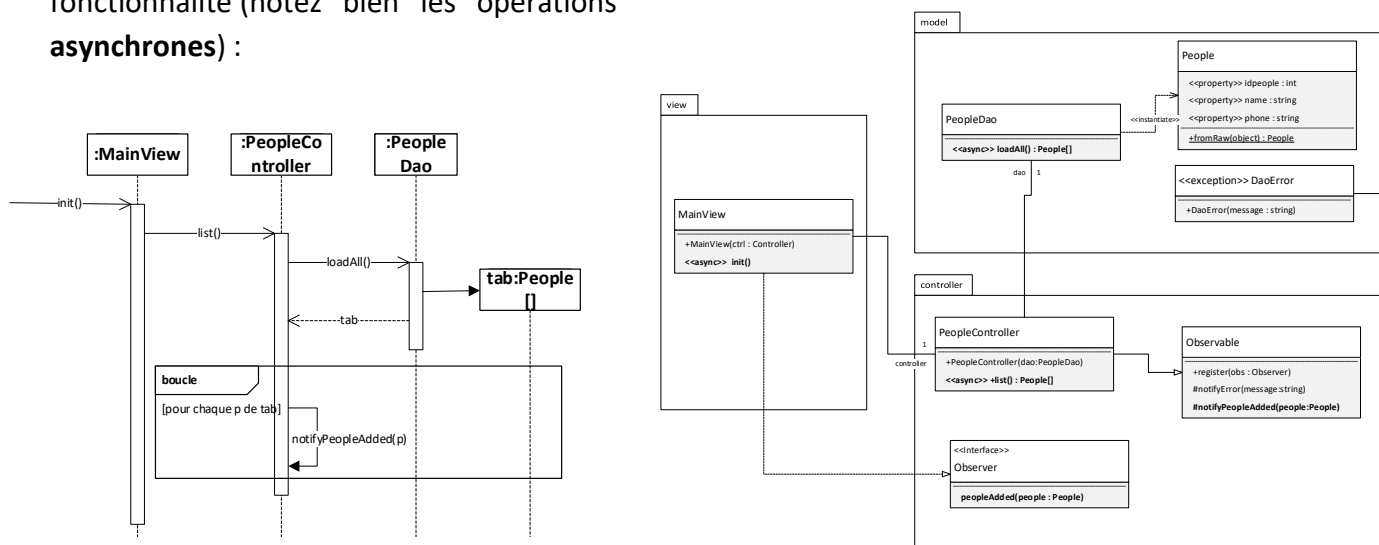
```
window.onload = ()=>{
  let dao = new PeopleDao();
  let ctrl = new PeopleController(dao);
  let view = new MainView(ctrl);
}
```

Ne pas oublier d'inclure les fichiers JS dans `index.html` (ou d'utiliser la notion de module).

Testez le site (il ne doit rien se passer mais aucune erreur sur la console) et validez (`git commit`) puis diffusez (`git push`) votre travail.

3 LISTER LES PERSONNES DE L'ANNUAIRE

Créez une branche « *list people* » et placez-vous dedans. Voici la description UML de cette fonctionnalité (notez bien les opérations **asynchrones**) :



3.1 MODÈLE

Créez l'opération `loadAll` dans la classe `PeopleDao`. Cette opération va utiliser (de manière asynchrone) la fonction `fetch` pour obtenir, depuis le service web, la liste des personnes. Levez l'exception `DaoError` en cas d'erreur.

Pour tester, il faut modifier le `main` temporairement et avoir :

```
let dao = new PeopleDao() ;
let result = await dao.loadAll() ;
console.log(result);
```

Vérifiez que le résultat contient bien les données présentes dans le serveur. Si c'est ok, validez.

3.2 CONTRÔLEUR

Rajoutez dans `Observer` et `Observable` les nouvelles opérations.

Rajoutez dans le contrôleur l'opération asynchrone `list` qui effectue les tâches suivantes :

- demande au dao la liste des personnes
- pour chaque personne, notifie la vue de son ajout
- en cas d'erreur, notifie avec l'opération `error`
- renvoie la liste

Pour tester, faites comme précédemment, en modifiant temporairement le `main` :

```
let dao = new PeopleDao() ;
let ctrl = new PeopleController(dao);
let result = await ctrl.list();
console.log(result);
```

vous devriez avoir la liste complète dans la console. Le test des notifications de la vue se fera après (il serait possible avec une vue « factice »). Si c'est ok, validez.

3.3 VUE

Il ne reste plus qu'à implémenter, dans la vue, les opérations `init` et `peopleAdded`. Avant cela, il faut déclarer un attribut de type `HTMLUListElement` et l'initialiser (avec `document.getElementById`) dans le constructeur.

Pour l'opération `peopleAdded`, il faut donc :

- Créer un élément « `li` » (avec `document.createElement`)
- Affecter à cet élément le texte issu du nom de la personne
- Créer un `span` contenant les classes « `id` » et « `hidden` » et dont le texte (`innerText`) est `idpeople`
- Ajouter ce `span` comme enfant de l'élément « `li` »
- Créer un `span` avec les classes « `phone` » et « `hidden` » et donc le contenu est `phone`
- Ajouter ce `span` comme enfant de l'élément « `li` »
- Ajouter cet élément comme enfant de la liste (en utilisant l'attribut)

Ne reste plus qu'à créer `init` qui appelle simplement la fonction `list` du contrôleur. Appelez `init` à la **fin** du constructeur de la vue et testez !

Vous pouvez supprimer du fichier HTML les `<li>` placés en exemple.

Si tout fonctionne, validez, poussez, fusionnez la branche avec la branche principale et passez à la suite.

4 SÉLECTIONNER UNE PERSONNE

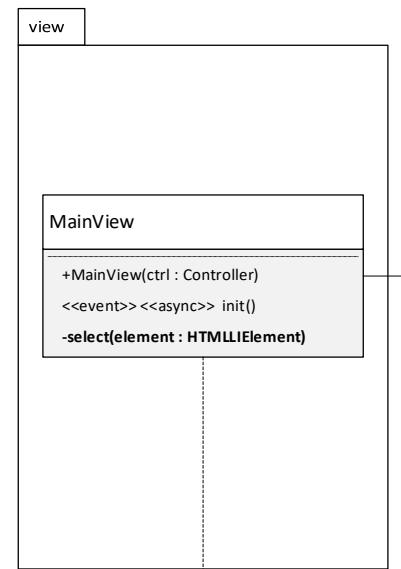
Créez une branche « *select people* » et placez-vous dedans.

Cette fonctionnalité servira simplement à permettre à une personne d'être sélectionnée, et ne concerne donc que la vue.

Le principe est le suivant : lorsqu'on **clique** sur une personne dans la liste (c'est-à-dire, sur un élément « `li` ») on souhaiterait que cet élément prenne la classe « `selected` » mais que tous les autres perdent cette classe.

Ajoutez l'opération indiquée en gras sur le diagramme ci-contre ; cette opération réalisera les travaux suivants :

- Retire la classe « `selected` » à tous les `<li>` contenus dans `#list`
- Ajoute la classe « `selected` » à l'élément passé en paramètre



Modifiez donc le code de `MainView.peopleAdded` pour que l'opération précédemment créée soit appelée lors d'un clic. Si tout fonctionne, validez, poussez et fusionnez la branche avec la branche principale.

Vous pouvez passer à la suite...

5 AFFICHAGE DU NUMÉRO DE TÉLÉPHONE

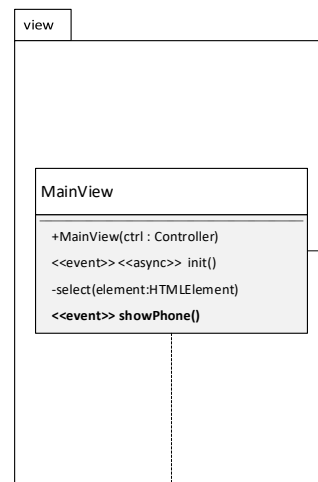
Créez une branche « *show phone* » et placez-vous dedans.

Cette fonctionnalité ne concerne que la vue et sert uniquement à afficher le numéro de téléphone de la personne sélectionnée.

Ajoutez l'opération indiquée en gras sur le diagramme ci-contre, qui réalisera :

- Une recherche (par `document.querySelector`) de l'enfant « `span` » du « `li` » sélectionné possédant la classe « `phone` »
- Récupération du texte interne au « `span` »
- Affiche de ce texte (le numéro de téléphone) avec `alert`

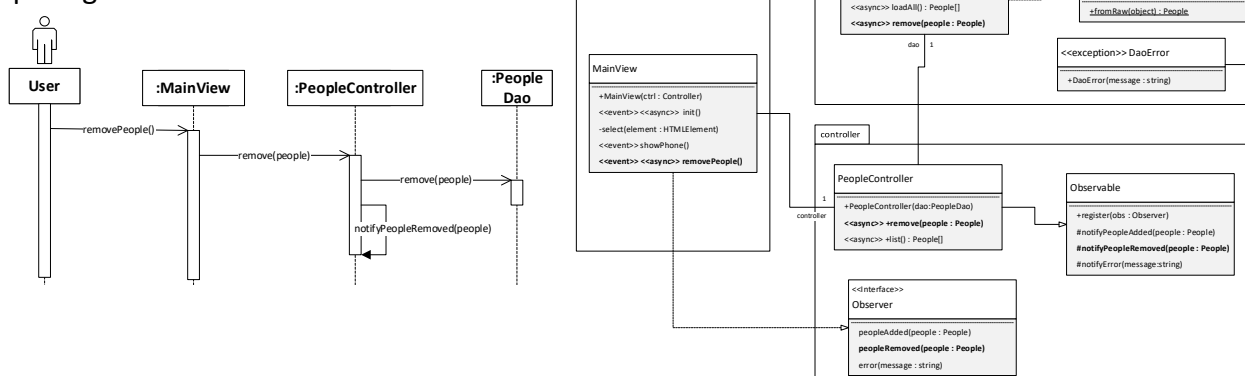
Reliez cette opération au clic sur le bouton « téléphone »¹. Si tout fonctionne, validez, poussez et fusionnez votre branche, puis passez à la suite !



6 SUPPRESSION D'UNE PERSONNE

Créez une branche « *remove people* » et placez-vous dedans.

Cette fonctionnalité consiste à supprimer une personne de l'annuaire partagé.



6.1 MODÈLE

Rajoutez dans le DAO l'opération pour la suppression. Cette opération devra utiliser le webservice (§ la documentation de ce dernier). N'oubliez pas d'utiliser `fetch` en asynchrone, de gérer les erreurs par une exception de type `DaoError`...

¹ Il pourrait être nécessaire de rajouter un id au bouton concerné, non ?

Pour tester, il suffit de retrouver l'identifiant d'une personne et de tester, toujours en modifiant temporairement le main (remplacez 494 par un id existant) :

```
let dao = new PeopleDao();
let p = People.fromRaw({"idpeople":494});
dao.remove(p);
```

Vérifiez en re-listant que la personne a bien disparu.

Validez votre dépôt.

6.2 CONTRÔLEUR

Rajoutez dans l'interface Observer et dans la classe Observable les notifications liées à la suppression.

Rajoutez l'opération dans le contrôleur qui utilise le dao et notifie de la suppression. N'oubliez pas que la moindre exception doit déclencher une notification à la vue.

Testez avec le code suivant dans une console (reprendre un nouvel ID évidemment) :

```
let dao = new PeopleDao();
let ctrl = new PeopleController(dao);
let p = People.fromRaw({"idpeople":494});
ctrl.remove(p);
```

Rafraichir pour vérifier que la suppression a bien eu lieu, validez.

6.3 VUE

Implémentée la nouvelle notification (personRemoved) dans la vue. Il faut pour cela chercher, parmi les « span » de classe « id », celui qui correspond à l'idpeople de la personne. Une fois trouvé, il faut supprimer son parent de la liste. Voici le code nécessaire :

```
let items = document.querySelectorAll("span.id");
for(let node of items) {
    if(node.innerHTML==people.idpeople.toString())
    {
        let parent = node.parentNode;
        this.list.removeChild(parent);
        break;
    }
}
```

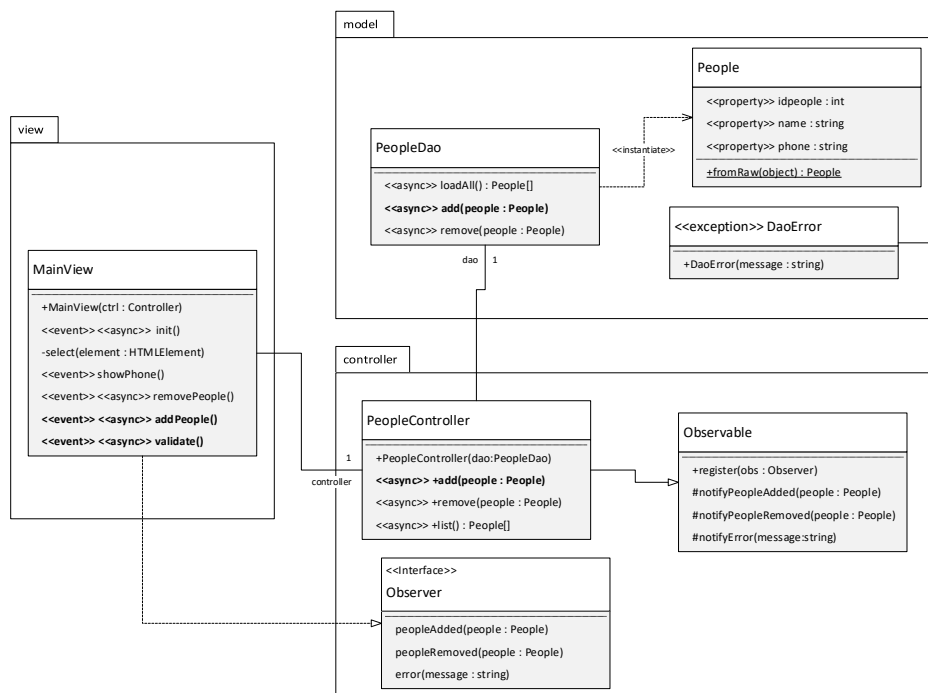
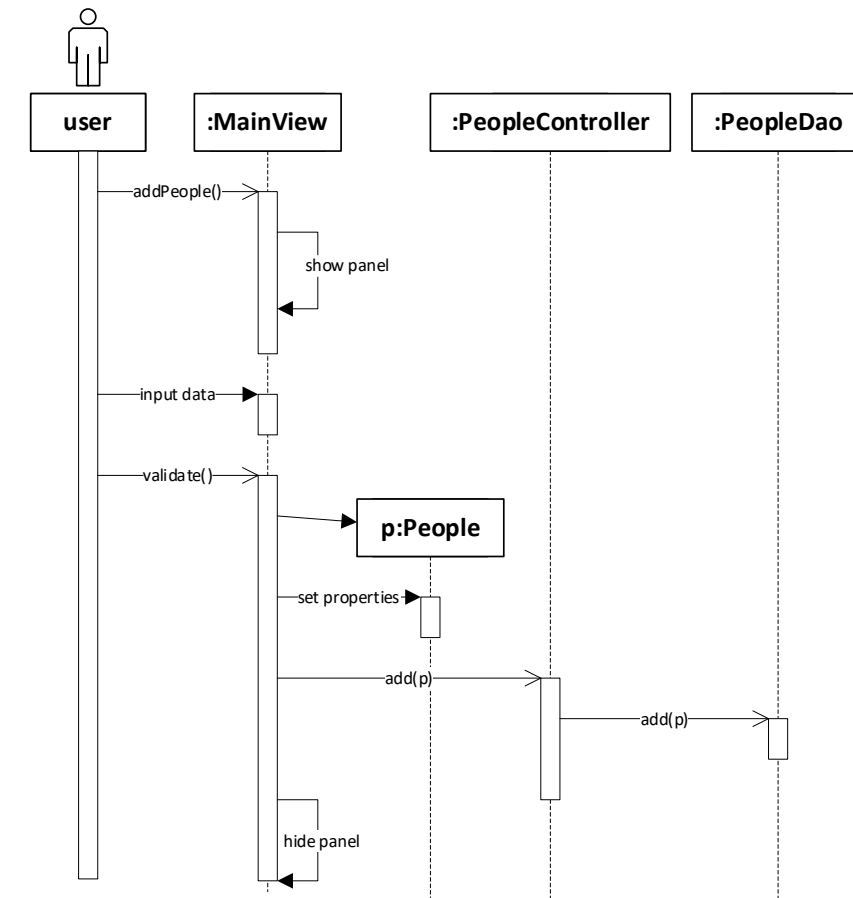
Rajoutez dans la vue l'opération removePeople, appelée par un clic sur le bouton « remove ». Cette opération doit :

- Récupérer l'id de la personne sélectionnée (voir question 0)
- Utiliser le contrôleur pour supprimer la personne sélectionnée

Testez. Si tout fonctionne, validez, publiez le code et fusionnez la branche avec la branche principale. Passez à la suite.

7 AJOUTER UNE PERSONNE

Créez une branche « *add people* » et placez-vous dedans.



7.1 MODÈLE

Rajoutez dans le dao l'opération add qui envoie au webservice (en POST) la personne reçue en paramètre. Il faudra donc la sérialiser, par exemple en utilisant `JSON.stringify`. En outre, il faudra désérialiser la réponse afin d'obtenir l'identifiant (`idpeople`) fixé par le serveur.

① Vous pouvez préciser les propriétés à utiliser en ajoutant un 2^{ème} paramètre à `stringify`, de type tableau de chaînes, contenant les noms des propriétés souhaitées.

Une fois l'application lancée, utilisez la modification manuelle du `main` pour ajouter « à la main » une personne (utilisez d'autres noms et numéro de téléphone que sur l'exemple car d'autres étudiants peuvent les utiliser) :

```
let p = new People(); p.name = "TEST"; p.phone = "666";
let dao = new PeopleDao(); dao.add(p);
```

Vérifiez que cette nouvelle personne vient bien d'être ajoutée et que l'identifiant est correct. Validez et continuez.

7.2 CONTRÔLEUR

Ajoutez l'opération add dans le contrôleur, qui utilise le dao et qui :

- En cas de succès notifie de l'ajout de cette personne
- En cas d'échec notifie d'une erreur

Testez simplement, avec le code suivant :

```
let p = new People(); p.name = "TEST"; p.phone = "666";
let dao = new PeopleDao(); let ctrl = new PeopleController(dao);
ctrl.add(p);
```

Si tout fonctionne, validez et continuez.

7.3 VUE

Il ne reste plus qu'à créer la vue. Celle-ci va comporter deux nouvelles opérations.

Commencez par l'opération `addPeople` qui, lors du clic sur le bouton ajouter, doit montrer la div ayant la classe « add » (qui est cachée au départ). Testez.

L'opération `validate`, quant à elle, effectue les opérations suivantes :

- Récupérer le nom et le numéro de téléphone saisis dans les `<input>`
- Créer une personne avec ces valeurs
- Utiliser le contrôleur pour ajouter cette personne
- Cacher la div de classe « add »

Testez ! Si tout est ok, il ne reste plus qu'à valider votre code, le pousser sur le dépôt distant et fusionner avec la branche principale.

8 FIN DU TP

Faites bien attention à ce que le code dans la branche principal soit 100% fonctionnel (n'y mettez pas une fonctionnalité en cours, non terminée).

Vérifiez bien que le dépôt GitHub est à jour, que votre enseignant a reçu son URL et qu'il est autorisé à consulter le dépôt, avant de quitter la salle.

